

SANCTUARY

Wellness Connect Platform

Comprehensive Technical Architecture & Code Integration Guide

A detailed reference documenting the implementation of Node.js / Express backend, React / Vite frontend, Mongoose schemas, and Google Gemini API integration.

DOCUMENT VERSION: 1.0.0

ENGINE: Google Gemini 2.5 Flash

DATE: June 2026

1. Architecture & Communication Flow

Sanctuary (Wellness Connect) is a modern, full-stack mental health application constructed to provide clients with interactive wellness support, private journaling, therapist tracking, and a community space. The stack is split into a modular backend and client frontend.

1.1 The Client Layer (Frontend)

Built using React 19 and Vite, the frontend operates as a single page application (SPA). Styling is handled via Tailwind CSS utilizing a custom theme mapping soothing, organic earth-tones (deep greens, soft stones, warm ambers). Axios is used for asynchronous HTTP networking, with interceptors bound to dynamically append authorization tokens stored in localStorage.

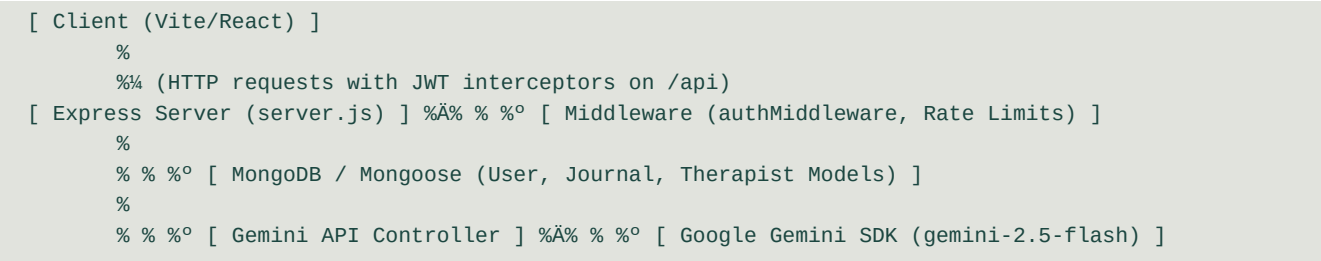
1.2 The Service Layer (Backend)

The backend is a Node.js REST API using the Express framework. It binds MongoDB using the Mongoose ODM. Features include Helmet security configurations to manage headers, CORS settings allowing frontend connectivity, rate limiters preventing request abuse, and custom JWT verification middleware to block unauthorized requests.

1.3 Gemini Integration Context

To provide active support, the backend directly communicates with the Google Gemini API. This is utilized for conversational therapy-like interaction (AI Companion Chat) and text parsing (Journal Analysis). The service leverages the gemini-2.5-flash model. Request/response flows route through controllers that process user credentials, validate access, filter history, and format payloads.

Flow Diagram: System Communication



2. Database Schema (Mongoose Models)

The database layers translate direct MongoDB documents into structured Javascript models via Mongoose. There are four primary schemas configured in the workspace.

2.1 User Schema (User.js)

Stores client identifiers, emails, encrypted passwords, and profile image links. The schema enforces uniqueness constraints.

```
const userSchema = new mongoose.Schema({
  username: { type: String, required: true, unique: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  profileImage: { type: String, default: '...' }
}, { timestamps: true });
```

2.2 Journal Schema with AI Integration (Journal.js)

Tracks private text entries written by the client. We expanded this model to save Gemini-generated insights (sentiment indicators, summaries, coping tips, and affirmations). Saving the analysis prevents redundant API costs.

```
const journalSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  title: { type: String, required: true },
  content: { type: String, required: true },
  mood: { type: String, enum: ['Happy', 'Sad', 'Anxious', 'Calm', 'Reflective', 'Neutral'], default: 'Neutral' },
  aiAnalysis: {
    sentiment: { type: String },
    summary: { type: String },
    copingTips: { type: [String] },
    encouragement: { type: String }
  }
}, { timestamps: true });
```

3. Backend Security & JWT Verification

Express APIs must secure user data, especially private journals. Sanctuary executes authorization via JSON Web Tokens (JWT) verified at request boundaries.

3.1 Token Verification Middleware (authMiddleware.js)

A verification middleware intercepts requests requiring credentials. It extracts the Bearer token from the Authorization header, decrypts it using a signature key, and loads the user model into the request object.

```
const protect = async (req, res, next) => {
  let token;
  if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
    try {
      token = req.headers.authorization.split(' ')[1];
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
      req.user = await User.findById(decoded.id).select('-password');
      next();
    } catch (error) {
      res.status(401).json({ message: 'Not authorized, token failed' });
    }
  }
  if (!token) {
    res.status(401).json({ message: 'Not authorized, no token' });
  }
};
```

3.2 Global Security Integrations (server.js)

Several defensive headers and configurations are attached within server.js:

- **Helmet.js:** Injects HTTP headers protecting the application from basic injection, clickjacking, and mime-type exploits.
- **CORS:** Manages cross-origin resource sharing. Restricts client connections to correct network addresses.
- **Rate Limiter:** Ensures a client can only request up to 100 requests per 15 minutes, preventing denial-of-service attempts.

4. Google Gemini API Integration

The core AI features are driven by `@google/generative-ai`. The API handles two use cases: AI Companion Chat (conversational) and Journal Analysis (structured sentiment & advice).

4.1 System Instructions & Model Choice

We use gemini-2.5-flash. In the chatbot environment, we supply System Instructions to establish boundaries, instructions, and tones. This keeps the assistant empathetic, conversational, safe, and distinct from medical therapists.

4.2 The Model-First History Bug & Solution

During initial deployment, the Gemini API returned a 400 Bad Request error. The Gemini chat engine expects conversation histories to alternate roles and strictly begin with a "user" role message. Since the frontend initialized with an assistant greeting ("model"), the history failed.

The solution was to filter the history array on the backend to locate the first user message, slicing out any leading model greeting before formatting the history. This ensures compatibility with the SDK.

```
// Locate the first user message, skip leading assistant greetings
const firstUserIndex = (history || []).findIndex((msg) => msg.role === 'user');
const filteredHistory = firstUserIndex !== -1 ? history.slice(firstUserIndex) : [];

const formattedHistory = filteredHistory.map((msg) => ({
  role: msg.role === 'assistant' ? 'model' : 'user',
  parts: [{ text: msg.content || '' }]
}));
```

4.3 Structured JSON Journal Analysis

To support rich rendering on the frontend, the journal analysis handler forces Gemini to return JSON data. We configure responseType to "application/json" and pass a target structure. To handle edge cases, the backend implements a try-catch string replacement fallback to strip codeblocks before parsing.

```
const model = genAI.getGenerativeModel({
  model: 'gemini-2.5-flash',
  generationConfig: { responseType: 'application/json' }
});
```

5. Frontend Components & User Flows

The frontend maps raw backend API responses into interactive pages. It uses a custom Tailwind setup mapping custom colors like "surface-container-low" and "primary".

5.1 AI Companion Chat Page (AiCompanionPage.jsx)

Implements a real-time conversational chat. Features include:

- **Starter Prompts:** Clickable button cards that allow clients to instantly trigger grounding scripts, mindfulness prompts, or breathing exercises.
- **Dynamic Scrolling:** Uses standard React useRef hooks paired with useEffect to scroll the container automatically as responses stream.
- **Advanced Design:** Bouncing elements display to represent the companion in a active thinking state.

5.2 Journal Analysis Presentation (JournalPage.jsx)

The private journal dashboard lists past entries. Under each card, a conditional block reads `journal.aiAnalysis`:

- **No Analysis:** Displays a clean button with a sparkle icon to "Get AI Insights", invoking a loading spinner during execution.
- **Analysis Present:** Displays the compiled insights: a sentiment flag, an empathetic summary paragraph, a list of coping strategies, and a styled highlight quotation representing positive encouragement.

Flow Diagram: Journal AI Analysis

```
[ Client clicks "Get AI Insights" ]
      %
      %% (POST /api/ai/analyze/:id)
[ Backend queries Journal DB ] % % %° [ Prompts Gemini (2.5-flash) ]
      %
      %% (JSON format result)
[ Frontend updates state & UI ] %Å% % [ Backend updates DB & returns entry ]
```